

Building a Unix Shell in C — Implementation Reference

A mechanism-first guide for the OS Mini-Project. Read once end-to-end, then flip back by section while coding.

How to use this. Every code block here is a mechanism demonstration, not a submission file. They're deliberately written as isolated snippets with hardcoded assumptions, no consistent naming scheme, and no error-path polish. Your job is to understand *why* each call is there, then write your own version with your own structure, naming, and error handling. If you paste any of this verbatim, it will look exactly like what it is.

§0 — Spec-to-section map

Spec item	Marks	Section
Display / prompt	10	§4
echo	4	§5.1
pwd	4	§5.2
cd	8	§5.3
history	9	§6
Foreground execution	10	§3, §7.1
Background execution	10	§7.2
I/O redirection	10	§8
Pipelining	15	§9
Signal handling	10	§10
Modularity + makefile	10	§11

Sections §1 (C refresher) and §2 (parser) carry no marks directly but everything below them depends on the data structures you settle on in §2. Don't skip it.

Build order recommendation: §4 prompt → §5 builtins → §3+§7.1 foreground → §7.2 background → §8 redirection → §9 pipes → §10 signals → §6 history → §11 modularity. History is late because it's self-contained and low-risk; signals are before it because signals interact with §7 and you want that settled while the execution code is fresh.

§1 — C refresher: exactly what this project needs

1.1 The `argv` shape

Every `exec` call wants the same shape: an array of `char *`, terminated by a `NULL` pointer.

```
char *argv[] = { "ls", "-l", "/tmp", NULL };
/* argv --> [ptr][ptr][ptr][NULL]
           |   |   |
           v   v   v
         "ls" "-l" "/tmp"          */
```

`char **argv` and `char *argv[]` mean the same thing as a function parameter. The `NULL` is not decoration — `execvp` has no length parameter, so the `NULL` is the *only* way it knows where the array ends. Forget it and `exec` reads off the end of your allocation into garbage.

For a dynamically built `argv`:

```
#define MAX_ARGS 64
char *argv[MAX_ARGS + 1]; /* +1 reserves the NULL slot */
int argc = 0;
/* token = one word from command (ls -l /tmp -> {"ls","-l","/tmp",NULL})
argv[argc++] = tok;      /* ... repeat while tokenizing ... */
argv[argc] = NULL;       /* ALWAYS, before every exec */
```

A fixed cap like this is fine for coursework as long as you bounds-check (`if (argc < MAX_ARGS)`). Growing with `realloc` is the alternative:

```
char **args = malloc(cap * sizeof *args);
if (!args) { perror("malloc"); return NULL; }
/* when full: */
char **tmp = realloc(args, (cap *= 2) * sizeof *args);
if (!tmp) { free(args); return NULL; } /* realloc failure does NOT free the original */
args = tmp;
/* note: there's no malloc cast -> forcing a type to a value while declaring it; like (char**) something
   malloc returns void *, and older C compilers assume undeclared functions return int, if stdlib.h is not included
```

That last comment is the classic `realloc` leak: `args = realloc(args, ...)` loses your only pointer to the old block if `realloc` returns `NULL`.

1.2 `strtok_r` and manual tokenizing

```
char *strtok_r(char *str, const char *delim, char **saveptr);
```

- First call: pass the string. Subsequent calls on the same string: pass `NULL`.
- Returns a pointer to the next token, or `NULL` when exhausted.

- It modifies the input string in place, overwriting each delimiter with `'\0'` and handing you a pointer into the middle of your buffer.

Three consequences that will bite you:

1. **Never call it on a string literal.** `strtok_r("ls -l", ...)` writes to read-only memory → segfault. Always tokenize a mutable buffer (e.g. the one `getline` gave you, or a `strdup` copy).
2. **Tokens are not independent.** They point into the original buffer. If you `free` the buffer, every token dangles. Either keep the buffer alive as long as the tokens, or `strdup` each token.
3. **`strtok` vs `strtok_r`:** plain `strtok` keeps its position in a hidden static variable, so you cannot have two tokenizations in flight. You will absolutely want nested tokenizing (split on `|`, then split each piece on spaces), so use `strtok_r` everywhere and give each level its own `saveptr`.

Runs of delimiters collapse automatically — `strtok_r` skips consecutive delimiters and never returns an empty token. That single behaviour is most of the `echo` spec for free.

```
char *save1 = NULL, *save2 = NULL;
for (char *seg = strtok_r(line, "|", &save1); seg; seg = strtok_r(NULL, "|", &save1)) {
    for (char *tok = strtok_r(seg, " \\t", &save2); tok; tok = strtok_r(NULL, " \\t", &save2)) {
        /* tok is one word of one pipeline stage */
    }
}
```

1.3 malloc/free discipline for this project

You only have four allocation lifetimes to worry about:

What	Allocated by	Freed when
The input line	<code>getline</code> (it mallocs for you)	end of each loop iteration, or reuse the buffer
<u>Parsed command struct</u>	your parser	end of each loop iteration
History strings	<code>strdup</code>	when evicted from the ring, and at exit
Duplicated tokens (if you copy)	<code>strdup</code>	with the command struct

The pattern that keeps this sane: **one `free_pipeline()` function that mirrors your `parse_pipeline()` exactly**. Every `malloc` in the parser gets a matching `free` in the destroyer, in reverse order. If you add a field to the struct, you edit both functions in the same sitting.

```
void free_pipeline(pipeline_t *p) {
    if (!p) return;
    for (int i = 0; i < p->ncmds; i++) {
        for (int j = 0; p->cmds[i].argv[j]; j++) free(p->cmds[i].argv[j]);
        free(p->cmds[i].argv);
        free(p->cmds[i].infile); /* free(NULL) is legal and does nothing */
        free(p->cmds[i].outfile);
    }
    free(p->cmds);
    free(p);
}
```

`free(NULL)` being a legal no-op is why you should `calloc` or explicitly `NULL`-initialise your structs — it removes every "is this field set?" check from the teardown path.

1.4 Structs for a parsed command

Design these before writing any execution code. Everything downstream reads them.

```
typedef struct {
    char **argv; /* NULL-terminated, argv[0] is the program name */
    char *infile; /* NULL if no '<' */
    char *outfile; /* NULL if no '>' */
    int append; /* optional: 1 if '>>' – not required by the spec */
} command_t;

typedef struct {
    command_t *cmds; /* array of ncmds stages */
    int ncmds; /* 1 = no pipe, 2 = one pipe, ... */
    int background; /* 1 if the line ended in '&' */
} pipeline_t;
```

This is the whole contract between your parser and your executor. `ncmds == 1` is not a special case — it's just a pipeline of length one, which means your pipeline code in §9 handles foreground execution too and you write the fork/exec logic exactly once.

Mechanism note. Notice `background` lives on the pipeline, not on a command. That's correct Unix semantics: `a | b | c &` backgrounds the *entire* pipeline as one job, not just `c`. Getting the struct shape right encodes the semantics for you.

§2 — Parsing: getting from a line to a `pipeline_t`

2.1 Reading the line

```
char *line = NULL;
size_t cap = 0;
ssize_t n;

print_prompt();
n = getline(&line, &cap, stdin);
```

`getline` mallocs (and reallocs) the buffer for you, so it handles arbitrarily long input. Pass the *same* `line/cap` pair back on the next call and it reuses the buffer.

Its return value is where two spec items live:

- `n > 0`: a line, **including the trailing `'\n'`** — strip it: `if (n > 0 && line[n-1] == '\n') line[n-1] = '\0';`
- `n == -1` **and** `feof(stdin)`: the user pressed **Ctrl+D** on an empty line → clean exit. (*Spec 3, signal handling*)
- `n == -1` **and** `errno == EINTR`: a signal interrupted the read → **not** EOF. Call `clearerr(stdin)` and loop back to a fresh prompt. This is exactly what happens on Ctrl+C. (*§10*)

Conflating those last two is the #1 reason student shells exit on Ctrl+C.

2.2 The metacharacter problem

`strtok_r` splits on delimiters but throws them away, which is fine for `|` (you already know where you split) but wrong for `<` and `>` where you need to know *which* token was the operator. Also, users write `ls>out.txt` with no spaces.

The cleanest fix is a **pre-pass that inserts spaces around every metacharacter**, then tokenize on whitespace only:

```
/* Returns a fresh buffer with ' ' padded around each of < > | & */
char *pad_metachars(const char *in) {
    size_t n = strlen(in);
    char *out = malloc(3 * n + 1); /* worst case: every char becomes " X " */
    if (!out) return NULL;
    size_t j = 0;
    for (size_t i = 0; i < n; i++) {
        if (strchr("<>|&", in[i])) {
            out[j++] = ' ';
            out[j++] = in[i];
            out[j++] = ' ';
        } else {
            out[j++] = in[i];
        }
    }
    out[j] = '\0';
    return out;
}
```

After this, `ls-l>out.txt|grep x` becomes `ls-l > out.txt | grep x` and a whitespace tokenizer handles everything. The `3*n+1` sizing is the important part: it's a proof that you cannot overflow, not a guess.

2.3 Assembling the pipeline

Sketch of the flow (write your own control structure, this is the shape):

1. Pad metacharacters, then split on `|` with `strtok_r` → gives you `ncmds` segments.
2. For each segment, tokenize on `"\t"`. Walk the token list with a small state check:
3. token is `<` → next token is `infile`, don't put either into `argv`
4. token is `>` → next token is `outfile`, don't put either into `argv`
5. token is `&` → set `background = 1`, don't put it into `argv`
6. otherwise → append to `argv`
7. `NULL`-terminate each `argv`.
8. If a segment produced zero `argv` entries (e.g. the user typed `ls |`), that's a parse error — report it and abort the whole line rather than forking a child with `argv[0] == NULL`.

Pitfalls here:

- `<` or `>` as the *last* token with no filename after it. Check for the missing operand instead of dereferencing `NULL`.
- `&` in the middle of a line (`sleep 5 & ls`). The spec only requires trailing `&`; simplest correct behaviour is to treat any `&` as "background the pipeline" and document that in your README.
- Empty input (user just hits Enter). Bail out before parsing — `ncmds == 0` should not reach the executor.

§3 — The process model: fork, exec, wait

This is the conceptual core of the whole assignment. Understand these three and Specs 3's 55 marks become mostly bookkeeping.

3.1 fork

```
pid_t fork(void);
```

Returns	Where
0	in the child
child's PID (> 0)	in the parent
-1, sets <code>errno</code>	in the parent, on failure (no child created)

`fork` creates a near-identical copy of the calling process: same code, same variables at the same values, same open file descriptors, same current directory. The **only** immediate difference is the return value, which is how a single piece of code takes two different paths.

Mechanism note — copy-on-write. The kernel does *not* actually duplicate your address space at fork time. It marks every page in both processes as read-only and shared. The first time either process *writes* to a page, the CPU raises a page fault, the kernel copies just that one 4 KB page, and both continue. This is why

forking a 2 GB process is fast, and why the child sees a snapshot that later diverges. It also explains why `fork` immediately followed by `exec` (the shell's entire job) is cheap — almost nothing gets copied before the address space is thrown away.

3.2 `exec`

```
int execlp(const char *file, char *const argv[]);
```

`exec` does not create a process. It **replaces the current process image** — code, data, stack, heap — with a new program, while keeping the same PID, the same open file descriptors, and the same working directory. That preservation of file descriptors across `exec` is precisely what makes redirection and pipes possible (§8, §9).

Return semantics are unusual and worth memorising: **on success, `execvp` never returns**. There is no code left to return to. So:

```
execvp(argv[0], argv);
/* reaching this line means exec FAILED */
perror(argv[0]);
_exit(127);
```

Any line after `execvp` is unconditionally an error path. If you forget it, a failed `exec` drops the child back into your shell's main loop and you now have two shells reading the same terminal — a confusing bug that looks like duplicated prompts.

Which variant to use:

- `execvp` — `v` = argv vector, `p` = search `PATH`. This is what you want; it's why `ls` works without typing `/bin/ls`.
- `execv` — no `PATH` search, needs a full path.
- `execlp`/`execl` — arguments as a variadic list, useless when you have an array.

Mechanism note — `_exit` vs `exit`. In a failed-`exec` child, use `_exit()`, not `exit()`. `exit()` flushes stdio buffers and runs `atexit` handlers — but the child inherited a *copy* of the parent's stdio buffers, so anything the parent had buffered but not yet written gets flushed a second time by the child. You get duplicated output that appears out of nowhere. `_exit()` terminates immediately without touching userspace buffers.

Mechanism note — exit code 127. By convention across Unix shells: 127 = command not found, 126 = found but not executable, 128+N = killed by signal N. Following it costs nothing and makes your shell behave like a real one under test scripts.

3.3 `wait` / `waitpid`

```
pid_t waitpid(pid_t pid, int *status, int options);
```

pid argument	Meaning
> 0	wait for exactly that child
-1	wait for any child
0	any child in the same process group

`options`: 0 blocks until the child changes state; `WNOHANG` returns 0 immediately if no child is ready — that's the flag that makes non-blocking reaping possible in §10.

Returns the reaped PID, 0 (with `WNOHANG`, nothing ready), or -1 on error — `errno == ECHILD` means "you have no such children", which is a normal condition, not a crash.

Decode `status` with macros, never by hand:

```
if (WIFEXITED(status)) printf("exited, code %d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status)) printf("killed by signal %d\n", WTERMSIG(status));
```

Mechanism note — zombies. When a child exits, the kernel cannot free its entry in the process table yet, because the parent may still want the exit status. So it keeps a stub — the **zombie** — holding just the PID and status. `wait` / `waitpid` collects it and the entry is released. A shell that forks background jobs and never waits accumulates zombies until it hits the process limit. That's the entire reason the spec's `SIGCHLD` requirement exists (§10.3).

3.4 The canonical skeleton

Every execution path in your shell is a variation on this. (Covers Spec 3, foreground execution, 10 marks.)

```
pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    return -1;
}

if (pid == 0) {
    /* ---- CHILD ---- */
    /* redirection / pipe wiring goes HERE, before exec (see §8, §9) */
    execvp(argv[0], argv);
    fprintf(stderr, "%s: command not found\n", argv[0]);
    _exit(127);
}

/* ---- PARENT ---- */
int status;
if (waitpid(pid, &status, 0) < 0) {
    if (errno != EINTR) perror("waitpid");
}
```

Note the `errno != EINTR` guard: a blocking `waitpid` can be interrupted by a signal (like your own `SIGCHLD` handler firing), returning -1. That's not an

error worth reporting. This is the tiny detail that separates a shell that prints spurious errors from one that doesn't.

\$4 — The prompt (Spec 1, 10 marks)

Three moving parts: username, hostname, and a path rewritten relative to the shell's home.

```
#include <unistd.h>
#include <pwd.h>
#include <sys/types.h>

/* Username. getpwuid returns a pointer to STATIC storage — the next call
overwrites it. strdup it if you intend to keep it. */
const char *username(void) {
    struct passwd *pw = getpwuid(getuid());
    return pw ? pw->pw_name : "unknown";
}

/* Hostname. HOST_NAME_MAX is not defined on macOS, so size it yourself. */
char host[256];
if (gethostname(host, sizeof host) != 0) {
    strcpy(host, "unknown");
}
host[sizeof host - 1] = '\0'; /* POSIX doesn't guarantee truncation is terminated */
```

`getenv("USER")` also works but is not equivalent — environment variables are user-modifiable and can be absent under `env -i` or in some non-login contexts. `getpwuid(getuid())` reads the actual passwd database, which is what "not hard-coded" really means.

4.1 The `~` substitution

The shell's home is the directory it was launched from, captured once at startup — not `$HOME`.

```
char shell_home[PATH_MAX];
if (!getcwd(shell_home, sizeof shell_home)) { perror("getcwd"); exit(1); }
```

Then, each time you print the prompt:

```
void display_path(const char *home, const char *cwd, char *out, size_t n) {
    size_t hl = strlen(home);
    if (strncmp(cwd, home, hl) == 0 && (cwd[hl] == '\0' || cwd[hl] == '/'))
        snprintf(out, n, "~%s", cwd + hl);
    else
        snprintf(out, n, "%s", cwd);
}
```

The `cwd[hl] == '\0' || cwd[hl] == '/'` check is the subtle bit. Without it, home `/home/user` would match cwd `/home/username_backup` and print `~name_backup`. You're testing for a path component boundary, not a string prefix.

Then:

```
printf("<%s@%s:%s> ", user, host, shown);
fflush(stdout); /* stdout is line-buffered; no '\n' means nothing appears without this */
```

The `fflush` is mandatory. Without a newline the prompt sits in the buffer and the user stares at a blank screen while your shell waits for input.

Pitfalls: call `getcwd` fresh on every prompt (it changes when `cd` runs); `getcwd` returns `NULL` if the current directory was deleted out from under you (`errno == ENOENT`) — handle it rather than passing `NULL` to `printf`.

\$5 — Built-in commands (Spec 2)

A built-in is any command your shell handles in its own process with no fork and no exec. The dispatch is a string comparison before you reach the execution path:

```
if (strcmp(argv[0], "cd") == 0) return builtin_cd(argv);
else if (strcmp(argv[0], "echo") == 0) return builtin_echo(argv);
else if (strcmp(argv[0], "pwd") == 0) return builtin_pwd(argv);
else if (strcmp(argv[0], "history") == 0) return builtin_history(argv);
else return -1; /* not a builtin — hand off to the external path */
```

Why `cd` must be a builtin. Not a style choice — a hard consequence of the process model. `chdir` changes the working directory of the calling process only. If `cd` forked, the child would change its directory and then exit, leaving the shell exactly where it was. Some commands are builtins for speed; `cd` is a builtin because there is no other way for it to work.

5.1 echo (4 marks)

The spec's whole requirement — collapse runs of spaces and tabs into one space — is already handled by your tokenizer, because `strtok_r` never returns empty tokens. If `echo all\t\tthe best` parsed into `argv = {"echo", "all", "the", "best", NULL}`, then:

```
int builtin_echo(char **argv) {
    for (int i = 1; argv[i]; i++) {
        if (i > 1) putchar(' ');
        fputs(argv[i], stdout);
    }
    putchar('\n');
    return 0;
}
```

The `if (i > 1)` prints separators *between* words rather than after each, which avoids a trailing space. Bare `echo` prints just a newline, which is correct.

5.2 `pwd` (4 marks)

```
int builtin_pwd(void) {
    char buf[PATH_MAX];
    if (!getcwd(buf, sizeof buf)) { perror("pwd"); return -1; }
    printf("%s\n", buf);
    return 0;
}
```

`getcwd(NULL, 0)` is a glibc extension that mallocs the right size for you — convenient, but then you must `free` it, and it's less portable. `PATH_MAX` from `<limits.h>` is fine here.

5.3 `cd` (8 marks)

Five cases plus an error case. The state you need is one string: the previous directory.

```
static char prev_dir[PATH_MAX];
static int have_prev = 0;

int builtin_cd(char **argv) {
    /* count args */
    int n = 0;
    while (argv[n]) n++;

    if (n > 2) { fprintf(stderr, "cd: too many arguments\n"); return -1; }

    const char *target;
    char resolved[PATH_MAX];

    if (n == 1 || strcmp(argv[1], "~") == 0) {
        target = shell_home; /* the launch dir from $4 */
    } else if (strcmp(argv[1], "-") == 0) {
        if (!have_prev) { fprintf(stderr, "No previous directory.\n"); return -1; }
        strncpy(resolved, prev_dir, sizeof resolved);
        resolved[sizeof resolved - 1] = '\0';
        target = resolved;
    } else {
        target = argv[1]; /* covers "..", relative and absolute paths */
    }

    /* Capture the current dir BEFORE moving — but only commit it if chdir succeeds. */
    char cur[PATH_MAX];
    if (!getcwd(cur, sizeof cur)) { perror("cd"); return -1; }

    if (chdir(target) != 0) { perror("cd"); return -1; }

    strncpy(prev_dir, cur, sizeof prev_dir);
    prev_dir[sizeof prev_dir - 1] = '\0';
    have_prev = 1;

    if (n == 2 && strcmp(argv[1], "-") == 0) {
        char now[PATH_MAX];
        if (getcwd(now, sizeof now)) printf("%s\n", now); /* spec: cd - prints where it went */
    }
    return 0;
}
```

Why `..` needs no special case: `chdir` resolves paths through the kernel's normal path walk, and `..` is a real directory entry in every directory. Special-casing it (string-chopping the `cwd`) actually gets you *wrong* answers in the presence of symlinks.

Pitfalls:

- Updating `prev_dir` before checking whether `chdir` succeeded — a failed `cd /nonexistent` must not clobber your `cd - target`.
- `cd -` when `prev_dir` points at a directory that has since been deleted: `chdir` fails, `perror` reports it, state stays consistent. That's the correct behaviour, and you get it free by ordering the code as above.
- `n > 2` is the "more than one argument" error the spec asks for — remember `argv[0]` is `"cd"` itself, so two arguments means `n == 3`.

\$6 — History (Spec 2, 9 marks)

Requirements, restated precisely: store the last 20, display the last 10, persist to a file in the shell's home (`~`), evict oldest past 20, and skip a command identical to the one immediately before it.

6.1 The ring buffer

A ring buffer (circular buffer) is a fixed array plus an index that wraps around with `%`. It gives you "keep the last N" without shifting elements.

```
#define HIST_CAP 20

static char *hist[HIST_CAP];
static int hist_count = 0; /* total commands ever added (monotonic) */
```

The trick that removes every off-by-one: let `hist_count` grow forever and always index with `% HIST_CAP`. The slot for the *k*-th command is `hist[k % HIST_CAP]`.

```
void hist_add(const char *cmd) {
    if (!cmd || !*cmd) return;
```

```

/* spec: skip exact duplicate of the immediately preceding command */
if (hist_count > 0) {
    const char *last = hist[(hist_count - 1) % HIST_CAP];
    if (last && strcmp(last, cmd) == 0) return;
}

int slot = hist_count % HIST_CAP;
free(hist[slot]); /* evicts the oldest; free(NULL) is safe on the first pass */
hist[slot] = strdup(cmd);
if (!hist[slot]) { perror("strdup"); return; }
hist_count++;
}

```

Displaying the last 10 is then a bounded walk backwards from the newest:

```

void hist_show(void) {
    int stored = hist_count < HIST_CAP ? hist_count : HIST_CAP;
    int show = stored < 10 ? stored : 10;
    int start = hist_count - show; /* index of the oldest one we'll print */

    for (int k = start; k < hist_count; k++)
        printf("%d %s\n", k - start + 1, hist[k % HIST_CAP]);
}

```

The off-by-one checklist — verify each by hand before you trust it:

- 0 commands stored → `show = 0`, loop doesn't run. OK
- 3 commands stored → prints all 3. OK
- Exactly 20 stored → prints commands 11–20. OK
- 25 stored → `hist_count = 25`, `start = 15`, prints 16–25, and slot `15 % 20 = 15` still holds command 16 (command 16 went to slot 15). OK

That last one is the check that catches a broken ring. Do it on paper.

6.2 Persistence

Path: the shell's home from §4, plus a dotfile name.

```

char hist_path[PATH_MAX];
snprintf(hist_path, sizeof hist_path, "%s/.myshell_history", shell_home);

```

`snprintf` (not `strcpy` + `strcat`) because it takes the buffer size and truncates instead of overflowing. Its return value tells you how many characters it wanted to write — if that's $\geq$ your buffer size, truncation happened.

Load at startup:

```

void hist_load(const char *path) {
    FILE *f = fopen(path, "r");
    if (!f) return; /* first run - absence is not an error */
    char *line = NULL; size_t cap = 0; ssize_t n;
    while ((n = getline(&line, &cap, f)) != -1) {
        if (n > 0 && line[n-1] == '\n') line[n-1] = '\0';
        hist_add(line); /* reuse the same path - dedup logic applies uniformly */
    }
    free(line);
    fclose(f);
}

```

Save: rewrite the whole file (oldest first) after each command, or once at exit.

```

void hist_save(const char *path) {
    FILE *f = fopen(path, "w"); /* "w" truncates - that IS the eviction mechanism */
    if (!f) { perror("history"); return; }
    int stored = hist_count < HIST_CAP ? hist_count : HIST_CAP;
    for (int k = hist_count - stored; k < hist_count; k++)
        fprintf(f, "%s\n", hist[k % HIST_CAP]);
    fclose(f);
}

```

Rewriting the whole file every command sounds wasteful — it is, but the file is at most 20 lines and it means a **crash or a `kill -9` never leaves you with a corrupt half-written history**. Append-only ("`a`") is faster but then the file grows past 20 and you need separate trimming logic. Choose the simple one.

Pitfalls:

- Saving only at exit means `Ctrl+D` works but a crash loses everything, and graders often test by killing the shell. Save per command.
- Adding the command to history *before* you check whether it's empty → blank entries.
- Whether `history` itself gets recorded is your call (bash records it). Just be consistent and say which in your README.
- Leaks: `free` every `hist[i]` on exit so a leak check comes back clean (§12).

§7 — Foreground and background execution (Spec 3, 10 + 10 marks)

7.1 Foreground

Foreground is the §3.4 skeleton plus one addition: **record the child's PID in a global before waiting**, because your `SIGINT` handler needs to know who to kill (§10).

```
extern volatile sig_atomic_t fg_pid; /* 0 = no foreground child */

int run_foreground(command_t *cmd) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return -1; }

    if (pid == 0) {
        /* Child: restore default signal dispositions before exec.
         * (See §10.2 – usually unnecessary, but explicit is safer.) */
        signal(SIGINT, SIG_DFL);
        apply_redirections(cmd); /* §8 */
        execvp(cmd->argv[0], cmd->argv);
        fprintf(stderr, "%s: command not found\n", cmd->argv[0]);
        _exit(127);
    }

    fg_pid = pid;
    int status;
    while (waitpid(pid, &status, 0) < 0) {
        if (errno == EINTR) continue; /* interrupted by a handler – keep waiting */
        perror("waitpid");
        break;
    }
    fg_pid = 0;
    return WIFEXITED(status) ? WEXITSTATUS(status) : -1;
}
```

The `while (... ) { if (errno == EINTR) continue; }` loop is the correct way to wait in a shell that has signal handlers. A handler firing mid-wait makes `waitpid` return `-1 / EINTR` even though the child is still running; retrying is the right response.

"Only one foreground command at a time" comes for free — the shell is blocked inside `waitpid` and isn't reading input.

7.2 Background

Identical up to the fork; the parent simply *doesn't* wait.

```
int run_background(command_t *cmd) {
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return -1; }

    if (pid == 0) {
        setpgid(0, 0); /* see below – this is the important line */
        signal(SIGINT, SIG_DFL);
        apply_redirections(cmd);
        execvp(cmd->argv[0], cmd->argv);
        fprintf(stderr, "%s: command not found\n", cmd->argv[0]);
        _exit(127);
    }

    setpgid(pid, pid); /* also in parent – avoids a race, see note */
    printf("[%d]\n", pid); /* optional but conventional */
    return 0;
}
```

Mechanism note — why `setpgid` is not optional here. When you press Ctrl+C, the terminal driver does not send SIGINT to "the shell". It sends it to every process in the terminal's **foreground process group**. A child created by `fork` inherits its parent's process group, so without `setpgid` your background `sleep 100` & sits in the shell's group and *dies on every Ctrl+C* — directly violating "Background processes must not be affected." `setpgid(0, 0)` puts the child in a brand-new group whose ID equals its own PID, and the terminal driver leaves it alone.

Why call `setpgid` in both processes. It's a deliberate race-avoiding idiom. After `fork`, the parent might reach `setpgid(pid, pid)` before the child runs at all, or the child might run first. Calling it in both means the group is set no matter who wins; the second call is a harmless no-op (or fails with `EACCES` if the child already exec'd, which you can ignore).

Pitfalls:

- A background child that reads from the terminal gets stopped with `SIGTSTIN` once it's in its own group. That's correct Unix behaviour and not a bug you need to fix for this spec.
- Not reaping background children → zombies. That's §10.3.
- Forgetting `fflush(stdout)` before forking: the child inherits a *copy* of the buffer, and both processes flush it later, so your text prints twice. Flush before every fork if you've printed anything without a newline.

§8 — I/O redirection (Spec 3, 10 marks)

8.1 The concept

Every process has a **file descriptor table**: a small array of integers mapping to open files. By convention `0` = stdin, `1` = stdout, `2` = stderr. Library calls like `printf` are just wrappers that ultimately `write(1, ...)`.

Redirection works because that table is **per-process, inherited across `fork`, and preserved across `exec`**. So the sequence is: fork a child, rearrange the child's table so fd 1 points at your file, then `exec`. The new program calls `write(1, ...)` exactly as it always does — it has no idea it isn't a terminal.

Mechanism note — two levels of indirection. An fd doesn't point at a file; it points at an **open file description** (a kernel object holding the file offset, access mode, and status flags), which in turn points at the file's inode. That double indirection is why `dup2(fd, 1)` makes fd and 1 *share a file offset* — writes through either advance the same position. It's also why two separate `open()` calls on the same file give you independent offsets, while `dup2` does not.

8.2 The calls


```
int open(const char *pathname, int flags, mode_t mode);
```

Redirection	Flags	Mode
< file	O_RDONLY	—
> file	O_WRONLY O_CREAT O_TRUNC	0644
>> file (optional)	O_WRONLY O_CREAT O_APPEND	0644

`mode` is only consulted when `O_CREAT` actually creates the file, and it's masked by the process `umask` (so `0644` typically lands as `0644` or `0640`). Returns the lowest unused fd, or `-1/errno`.

```
int dup2(int oldfd, int newfd);
```

Makes `newfd` refer to the same open file description as `oldfd`, **atomically closing `newfd` first if it was open**. Returns `newfd`. If `oldfd == newfd`, it does nothing (and notably does *not* close it) — an edge case that matters in §9.

8.3 Worked example

(Covers Spec 3 I/O redirection, 10 marks.)

```
/* Called in the CHILD only, after fork, before exec. */
static int apply_redirections(command_t *cmd) {
    if (cmd->infile) {
        int fd = open(cmd->infile, O_RDONLY);
        if (fd < 0) { perror(cmd->infile); return -1; }
        if (dup2(fd, STDIN_FILENO) < 0) { perror("dup2"); close(fd); return -1; }
        close(fd); /* the duplicate at fd 0 keeps the file open */
    }

    if (cmd->outfile) {
        int flags = O_WRONLY | O_CREAT | (cmd->append ? O_APPEND : O_TRUNC);
        int fd = open(cmd->outfile, flags, 0644);
        if (fd < 0) { perror(cmd->outfile); return -1; }
        if (dup2(fd, STDOUT_FILENO) < 0) { perror("dup2"); close(fd); return -1; }
        close(fd);
    }
    return 0;
}
```

Then in the child: `if (apply_redirections(cmd) < 0) _exit(1); execvp(...)`. Bailing out with `_exit` rather than continuing is important — running `grep` with a failed input redirect would silently read the keyboard instead.

Reading `grep "hello" < input.txt > output.txt` through this: the parser strips `< input.txt` and `> output.txt`, leaving `argv = {"grep", "\"hello\"", NULL}` and both filenames on the struct. `grep` itself never sees the redirections — that's the whole point. (The spec says you needn't handle quotes, so `"hello"` with literal quote characters reaching `grep` is acceptable.)

Pitfalls:

- **Doing this in the parent.** `dup2(fd, 1)` in the shell process redirects *your shell's* stdout permanently and every prompt afterwards disappears into the file. It must happen after `fork`, in the child branch.
- Not closing `fd` after `dup2`. You now have two descriptors on the file; harmless for one command, but it leaks a descriptor per command and eventually you hit the per-process limit.
- Truncation timing: `O_TRUNC` empties the file at `open` time, which is *before* `exec`. `cat file > file` therefore empties the file before `cat` ever reads it. Real shells behave identically — this is not a bug you need to fix.
- Redirection interacting with pipes: apply redirections *after* pipe wiring so an explicit `>` on the last stage overrides the pipe. See §9.4.

§9 — Pipelining (Spec 3, 15 marks)

9.1 What a pipe actually is

```
int pipe(int fds[2]);
```

`pipe` creates a kernel-managed buffer (typically 64 KB) and hands you two descriptors onto it: `fds[0]` is the **read** end, `fds[1]` is the **write** end. Data written to `[1]` comes out of `[0]`, first in first out. Nothing touches the filesystem.

Mnemonic: **0 is stdin-ish (read), 1 is stdout-ish (write)** — same as the fd numbers themselves.

Two behaviours make the whole thing work, and both are about blocking:

- A `read` on an empty pipe **blocks** until data arrives, and returns `0` (EOF) **only when every write end has been closed**.
- A `write` to a full pipe blocks until the reader drains it.

That first rule is the source of nearly every pipe bug. The kernel counts how many descriptors reference the write end *across all processes*. If your shell forgets to close its copy, that count never drops to zero, `grep` never sees EOF, and your shell hangs forever waiting for a child that's waiting for data that will never stop coming.

Mechanism note — SIGPIPE. The reverse case: writing to a pipe whose read ends are all closed sends the writer `SIGPIPE`, which by default kills it. That's why `yes | head -5` terminates instead of running forever — `head` exits, closes the pipe, and `yes` dies. Do **not** set `SIGPIPE` to ignore in your shell's children, or commands like this will hang.

9.2 Complete two-command pipeline

(Covers Spec 3 pipelining, 15 marks — this is the code to understand line by line.)

```
/* Runs: cmdA | cmdB e.g. ls -l | grep "txt" */
int run_two_stage(char **argvA, char **argvB) {
    int fd[2];
    if (pipe(fd) < 0) { perror("pipe"); return -1; }

    pid_t p1 = fork();
    if (p1 < 0) { perror("fork"); close(fd[0]); close(fd[1]); return -1; }

    if (p1 == 0) {
        /* ---- LEFT child: writes into the pipe ---- */
        close(fd[0]); /* not reading — must close or B may never see EOF */
        dup2(fd[1], STDOUT_FILENO); /* stdout now goes into the pipe */
        close(fd[1]); /* the copy at fd 1 keeps it alive */
        execvp(argvA[0], argvA);
        perror(argvA[0]);
        _exit(127);
    }

    pid_t p2 = fork();
    if (p2 < 0) { perror("fork"); close(fd[0]); close(fd[1]); return -1; }

    if (p2 == 0) {
        /* ---- RIGHT child: reads from the pipe ---- */
        close(fd[1]); /* CRITICAL: if this stays open, read() never returns 0 */
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execvp(argvB[0], argvB);
        perror(argvB[0]);
        _exit(127);
    }

    /* ---- PARENT: holds no useful ends. Close BOTH. ---- */
    close(fd[0]);
    close(fd[1]);

    int status;
    waitpid(p1, &status, 0);
    waitpid(p2, &status, 0); /* the pipeline's status is the LAST command's */
    return WIFEXITED(status) ? WEXITSTATUS(status) : -1;
}
```

Count the descriptors. After `pipe()` there is 1 copy of each end. After two forks there are 3 copies of each end (parent + two children). The pipeline only terminates if the write-end count reaches 0, so all three write-end copies must be closed: the left child's is consumed by the `dup2` chain, the right child closes it explicitly, and the parent closes it explicitly. Miss any one and you hang. **When your pipeline hangs, the answer is always an unclosed descriptor.**

Ordering rule inside the child: `dup2` first, then close the original. Doing it in the other order closes the only reference to the pipe before you've duplicated it.

9.3 Generalising to N stages

The pattern: carry the *read end of the previous pipe* forward in a variable, and create a new pipe only when there's a stage to the right.

```
int run_pipeline(pipeline_t *pl) {
    int prev_read = -1; /* -1 = first stage, stdin unchanged */
    pid_t pids[MAX_STAGES];

    for (int i = 0; i < pl->ncmds; i++) {
        int fd[2] = { -1, -1 };
        int last = (i == pl->ncmds - 1);

        if (!last && pipe(fd) < 0) { perror("pipe"); return -1; }

        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return -1; }

        if (pid == 0) {
            /* ---- CHILD i ---- */
            if (prev_read != -1) { /* not the first: read from the previous pipe */
                dup2(prev_read, STDIN_FILENO);
                close(prev_read);
            }
            if (!last) { /* not the last: write into the new pipe */
                close(fd[0]);
                dup2(fd[1], STDOUT_FILENO);
                close(fd[1]);
            }
            apply_redirections(&pl->cmds[i]); /* AFTER wiring — explicit < > wins */
            execvp(pl->cmds[i].argv[0], pl->cmds[i].argv);
            fprintf(stderr, "%s: command not found\n", pl->cmds[i].argv[0]);
            _exit(127);
        }

        /* ---- PARENT: close what it no longer needs, carry the read end forward ---- */
        if (prev_read != -1) close(prev_read);
        if (!last) {
            close(fd[1]); /* parent never writes */
            prev_read = fd[0]; /* next iteration's child will read from here */
        }
        pids[i] = pid;
    }

    int status = 0;
    for (int i = 0; i < pl->ncmds; i++) waitpid(pids[i], &status, 0);
    return WIFEXITED(status) ? WEXITSTATUS(status) : -1;
}
```

```
}
```

Trace `a | b | c` through it:

i	pipe created	child's stdin	child's stdout	parent afterwards
0 (a)	P1	terminal	P1 write	closes P1 write, keeps P1 read
1 (b)	P2	P1 read	P2 write	closes P1 read <i>and</i> P2 write, keeps P2 read
2 (c)	none	P2 read	terminal	closes P2 read

Every end reaches zero references at exactly the right moment. Notice the parent's closes happen *after* the fork, which is what makes the child's inherited copy the surviving one.

Note that `ncmds == 1` runs through this loop untouched: no pipe is created, `prev_read` stays `-1`, and it degenerates into a plain fork/exec/wait. **That's why building your executor around the pipeline function means you write fork/exec once instead of three times.**

9.4 Redirection + pipes together

Apply redirections *after* pipe wiring (as above) so that in `cat < in.txt | grep x > out.txt`:

- stage 0's `dup2(fd[1], 1)` is overwritten by nothing (it has no `>`), and its `<` sets stdin — correct;
- stage 1's `dup2(prev_read, 0)` sets stdin from the pipe, then its `>` overwrites stdout — correct.

If you reversed the order, the pipe would clobber the user's explicit redirect and `out.txt` would stay empty.

9.5 Pitfalls summary

- Unclosed write end anywhere → hang.** First thing to check.
- Closing `prev_read` in the parent *before* forking the child that needs it.
- Forking all children then waiting: correct, and required. Forking one, waiting, then forking the next deadlocks as soon as the first command writes more than the pipe buffer (64 KB), because it blocks on a full pipe that nobody is draining.
- `fd[0]` / `fd[1]` mixed up. Read = 0, write = 1.
- Not reaping every stage — you get zombies per pipeline, not just per job.
- Backgrounded pipelines (`a | b &`): fork all stages, `setpgid` them into one group, don't wait.

§10 — Signal handling (Spec 3, 10 marks)

The hardest 10 marks in the assignment, because signal handlers run under rules that nothing else in C does.

10.1 What a signal is, mechanically

A signal is an **asynchronous interrupt delivered to a process**. When one arrives, the kernel suspends whatever the process was doing — possibly halfway through a `malloc`, or between two instructions of a `printf` — pushes a frame onto the stack, runs your handler, and then resumes the interrupted code.

Two consequences follow directly and they drive every rule below:

- Your handler can run at any point in your program**, including inside a library function that isn't reentrant.
- Signals are not queued.** The kernel tracks "SIGCHLD is pending" as a single bit. If three children exit while SIGCHLD is blocked, you get **one** delivery, not three. This is why §10.3 loops.

10.2 Installing handlers with `sigaction`

```
struct sigaction {
    void (*sa_handler)(int);
    sigset_t sa_mask;          /* extra signals blocked while the handler runs */
    int sa_flags;
};
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

Use `sigaction`, not `signal()`. `signal()`'s semantics vary between systems (on some, the handler resets to default after firing once) and it gives you no control over restart behaviour.

```
static void install(int sig, void (*fn)(int), int flags) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = fn;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = flags;
    if (sigaction(sig, &sa, NULL) < 0) { perror("sigaction"); exit(1); }
}

/* In main(), before the read loop: */
install(SIGINT, sigint_handler, 0);          /* NO SA_RESTART — see below */
install(SIGCHLD, sigchld_handler, SA_RESTART); /* WITH SA_RESTART — see below */
```

The `SA_RESTART` decision is deliberate, not cosmetic. When a signal interrupts a blocking syscall like the `read` underneath `getline`:

- Without `SA_RESTART`: the syscall returns `-1` with `errno == EINTR`. For `SIGINT` that's exactly what you want — the read aborts, you notice, and you print a fresh prompt. That's the spec's "print a new prompt".
- With `SA_RESTART`: the kernel silently restarts the syscall. For `SIGCHLD` that's what you want — a background job finishing shouldn't disturb the line the user is typing.

Mechanism note — handler inheritance across fork and exec. A forked child inherits the parent's handlers. On `exec`, handlers set to a *function* are reset to `SIG_DFL` (the new program's code doesn't exist in the old address space, so the pointer would be meaningless), but signals set to `SIG_IGN` stay ignored. Practical upshot: if you handle `SIGINT` with a function, `exec'd` children get default behaviour automatically. If you ever `signal(SIGINT, SIG_IGN)` in the shell, every child inherits immunity to `Ctrl+C` and you can never kill anything.

10.3 What is safe inside a handler

Only **async-signal-safe** functions — a specific POSIX-defined list. The important members: `write`, `_exit`, `kill`, `waitpid`, `signal`, `sigaction`.

The important *non*-members: `printf` and the whole `stdio` family, `malloc`/`free`, `strdup`, `perror`.

Why `printf` is genuinely dangerous, not just discouraged: `printf` writes into a shared `stdio` buffer guarded by a lock, and may call `malloc`. If a signal arrives while the main program is inside `printf` holding that lock, and your handler calls `printf` too, it deadlocks against a lock held by itself. It usually works. It occasionally hangs the shell during a demo. Use `write(STDOUT_FILENO, "\n", 1)` instead — it's a single syscall with no userspace state.

The other rule: **the only variable type you can safely share** between a handler and the main program is `volatile sig_atomic_t`.

- `sig_atomic_t` is an integer type the platform guarantees is read and written in a single uninterruptible instruction, so you never observe a half-updated value.
- `volatile` stops the compiler from caching it in a register — without it, a loop like `while (!flag) {}` can be optimised into an infinite loop because the compiler proves nothing in the loop modifies `flag`.

`pid_t` is `int` on Linux and macOS so storing a PID in a `sig_atomic_t` is fine in practice; cast it back when calling `kill`.

Also: **save and restore `errno`** in every handler. If the handler runs between a failed syscall and the main code's `errno` check, and the handler's own `waitpid` overwrites `errno`, you get a bogus error message from an unrelated place.

10.4 SIGINT — Ctrl+C

Requirements: shell survives, a fresh prompt appears, the foreground child (if any) dies, background jobs are untouched.

```
volatile sig_atomic_t fg_pid = 0;
volatile sig_atomic_t got_sigint = 0;

static void sigint_handler(int sig) {
    (void)sig;
    int saved = errno;

    if (fg_pid > 0)
        kill((pid_t)fg_pid, SIGINT);    /* kill() is async-signal-safe */

    write(STDOUT_FILENO, "\n", 1);      /* write() is safe; printf() is not */
    got_sigint = 1;                     /* let the main loop redraw the prompt */

    errno = saved;
}
```

Main loop side:

```
for (;;) {
    print_prompt();
    ssize_t n = getline(&line, &cap, stdin);

    if (n == -1) {
        if (errno == EINTR && got_sigint) { /* Ctrl+C, not EOF */
            got_sigint = 0;
            clearerr(stdin);                /* clear the error flag or the next read fails too */
            errno = 0;
            continue;                       /* loop → fresh prompt: the spec's requirement */
        }
        if (feof(stdin)) {                 /* Ctrl+D on an empty line */
            printf("\n");
            break;                          /* clean exit: save history, free memory */
        }
        perror("getline");
        break;
    }
    /* ... normal processing ... */
}
```

`clearerr(stdin)` is easy to miss: once a `FILE *` has its error flag set, every subsequent read fails immediately and your shell spins printing prompts forever.

Mechanism note — the child usually dies anyway. The terminal driver sends `SIGINT` to the entire foreground process group, and your foreground child is in that group, so it already receives `Ctrl+C` without your `kill()`. So why call it? Two reasons: it's what the spec asks you to demonstrate, and it makes your shell correct if you later add proper job control with `setpgid` on foreground children too. The extra `kill` is harmless (it can only fail with `ESRCH` if the child already died).

10.5 SIGCHLD — reaping background children

```
static void sigchld_handler(int sig) {
    (void)sig;
    int saved = errno;
    pid_t p;
    int status;

    while ((p = waitpid(-1, &status, WNOHANG)) > 0) {
        /* Record only; do NOT printf here. */
        if (n_reaped < MAX_REAPED) {
            reaped_pids[n_reaped] = p;
        }
    }
}
```

```

        reaped_stat[n_reaped] = status;
        n_reaped++;
    }
    }
    errno = saved;
}

```

Why the `while` loop, not a single `waitpid`: because signals aren't queued (§10.1). Three background jobs finishing at once produce one `SIGCHLD`. A handler that reaps one child leaves two zombies that are never collected. The loop drains everything currently reaped; `WNOHANG` makes it return `0` and exit the loop when nothing more is ready.

Then in the main loop, *before* printing each prompt, drain the record and print with normal `printf` — outside handler context, all functions are safe again:

```

void report_finished(void) {
    while (n_reaped > 0) {
        n_reaped--;
        printf("[%d] done\n", (int)reaped_pids[n_reaped]);
    }
}

```

The self-pipe trick. That shared array is still technically racy — the handler can fire mid-update in the main loop. The standard industrial fix is the *self-pipe trick*: at startup, create a pipe; the handler does nothing but `write(pipe_wr, &byte, 1)` (one safe syscall, atomic for small writes); the main loop `select()`s or reads on the read end and does all the real work there. You don't need it for this assignment — the bounded array is fine at coursework scale — but knowing the name and the shape is worth more than the marks.

10.6 The `SIGCHLD`-vs-foreground-wait bug

This one catches almost everyone, so read it twice.

Your foreground path calls `waitpid(fg_pid, &status, 0)`. Your `SIGCHLD` handler calls `waitpid(-1, ..., WNOHANG)` — and **-1 means any child, including the foreground one**. When the foreground child exits, `SIGCHLD` fires, the handler reaps it first, and the main `waitpid` then returns `-1` with `errno == ECHILD` because that child no longer exists. Symptoms: spurious error messages, lost exit statuses, or a shell that returns to the prompt before the command's output appears.

The clean fix is to **block `SIGCHLD` around the foreground fork-and-wait**:

```

sigset_t mask, prev;
sigemptyset(&mask);
sigaddset(&mask, SIGCHLD);

sigprocmask(SIG_BLOCK, &mask, &prev); /* SIGCHLD now held pending, not delivered */

pid_t pid = fork();
if (pid == 0) {
    sigprocmask(SIG_SETMASK, &prev, NULL); /* child starts with a clean mask */
    /* ... redirections, exec ... */
}

fg_pid = pid;
int status;
while (waitpid(pid, &status, 0) < 0 && errno == EINTR) ; /* reaps it ourselves */
fg_pid = 0;

sigprocmask(SIG_SETMASK, &prev, NULL); /* any pending SIGCHLD is delivered NOW */

```

Blocking a signal doesn't stop `waitpid` from reaping — it only defers *delivery of the notification*. Background children that exit during this window queue up their (single) pending `SIGCHLD`, and the handler's `while` loop cleans all of them the instant you unblock. Everything stays consistent.

The signal mask is also inherited across `fork` and `exec`, which is why the child restores `prev` — otherwise you'd hand `ls` a shell with `SIGCHLD` mysteriously blocked.

10.7 Signal pitfalls checklist

- `printf` in a handler → possible deadlock. Use `write`.
- No `volatile` on a shared flag → the optimiser deletes your check.
- Single `waitpid` in the `SIGCHLD` handler → zombies survive.
- Not saving/restoring `errno` → phantom error messages elsewhere.
- Missing `clearerr(stdin)` after `EINTR` → infinite prompt loop.
- Treating `getline() == -1` as unconditional EOF → shell exits on `Ctrl+C`.
- `SIG_IGN` for `SIGINT` → children inherit it and become unkillable.
- No `setpgid` on background children → `Ctrl+C` kills them, violating the spec.

§11 — Modularity and the makefile (Spec 4, 10 marks)

11.1 File split

The spec asks for logical separation, and the grader is reading for it. A split that maps to the spec:

File	Contents
<code>main.c</code>	the read–parse–execute loop, startup and shutdown
<code>prompt.c/.h</code>	username, hostname, ~ substitution (Spec 1)

parse.c/.h	tokenizer, metachar padding, <code>pipeline_t</code> construction, <code>free_pipeline</code>
builtins.c/.h	dispatch + <code>echo</code> , <code>pwd</code> , <code>cd</code> (Spec 2)
history.c/.h	ring buffer, load, save (Spec 2)
execute.c/.h	fork/exec, foreground, background, redirection, pipelines (Spec 3)
signals.c/.h	handler installation and handlers (Spec 3)
shell.h	shared types (<code>command_t</code> , <code>pipeline_t</code>) and shared constants

Header hygiene, since it's cheap marks:

```
#ifndef HISTORY_H
#define HISTORY_H
void hist_add(const char *cmd);
void hist_show(void);
#endif
```

Include guards prevent a type being defined twice when two headers include a third. Headers declare; `.c` files define. Anything not used outside its own file gets `static` — it keeps the linker namespace clean and signals intent to a reader.

Globals like `fg_pid` live in exactly one `.c` (the definition) and appear in the header as `extern volatile sig_atomic_t fg_pid`; . Defining it in the header instead gives you duplicate-symbol link errors.

11.2 makefile

Name it `makefile` exactly as the submission format specifies. **Recipe lines must begin with a real tab character, not spaces** — this is the single most common reason a makefile fails, and the error message (missing separator) doesn't say so.

```
CC      = gcc
CFLAGS  = -Wall -Wextra -g
TARGET  = shell
SRCS    = main.c prompt.c parse.c builtins.c history.c execute.c signals.c
OBJS    = $(SRCS:.c=.o)

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJS) $(TARGET)

.PHONY: all clean
```

Reading the automatic variables: `$@` is the target being built, `$^` is all prerequisites, `$<` is the first prerequisite. `$(SRCS:.c=.o)` is a substitution reference that rewrites the extension of every item in the list.

`.PHONY` tells make that `all` and `clean` are not filenames. Without it, a stray file named `clean` in your directory makes `make clean` report "nothing to be done" — a genuinely confusing five-minute bug.

`clean` should remove the `.o` files as well as the executable, since your submission is required to be flat (no subdirectories) and object files would otherwise ship with it.

\$12 — Pre-submission checklist

Compile clean

- ☐ `make clean` && `make` from scratch, on a machine you haven't been developing on if possible.
- ☐ Zero output from `gcc -Wall -Wextra`. Warnings you're tempted to ignore — unused variable, comparison between signed and unsigned, implicit declaration — are the exact class that becomes a segfault at grading time. `implicit declaration of function` in particular means a missing `#include` and the return value is being treated as `int`, which breaks on 64-bit pointers.
- ☐ Optionally `-Wall -Wextra -Werror` during development to force yourself to fix them.

Built-ins

- ☐ `echo`, `echo hello`, `echo multiple spaces` and with tabs → single spaces.
- ☐ `pwd` matches the system `pwd`.
- ☐ `cd` bare, `cd ~`, `cd ..`, `cd -` twice in a row (should toggle), `cd -` as the very first command (No previous directory.), `cd /nonexistent`, `cd a b` (too many arguments).
- ☐ `cd ..` from `/` doesn't crash.

History

- ☐ Fewer than 10 commands → prints only what exists.
- ☐ More than 20 → oldest evicted, `history` shows the last 10.
- ☐ The same command twice in a row is stored once; the same command with something in between is stored twice.
- ☐ Exit, relaunch, `history` → previous session's commands are there.
- ☐ The history file lands in the shell's launch directory, not `$HOME`.

Processes and signals

- `ls -l`, `sleep 3` (prompt waits), `sleep 3 &` (prompt returns immediately).
- `Ctrl+C` at an empty prompt → new prompt, shell alive.
- `Ctrl+C` during `sleep 100` → child dies, shell alive.
- `sleep 100 &` then `Ctrl+C` → **background job still alive** (`ps` to confirm). This is the `setpgid` test.
- Run 5 background jobs, let them finish, then `ps aux | grep defunct` → no zombies.
- `Ctrl+D` on an empty line → clean exit. `Ctrl+D` on a non-empty line → should not exit.
- `nonexistentcommand` → error message, shell survives, prompt returns.

Redirection and pipes

- `ls > out.txt`, `wc -l < out.txt`, `grep x < in.txt > out.txt`.
- `>` to a path in a nonexistent directory → error, no crash.
- `ls -l | grep txt`, `cat file | grep x | wc -l` (multi-stage), `ls | head -2` (early reader exit — must not hang).
- `cat bigfile | wc -l` with a file over 64 KB — catches the fork-then-wait-then-fork deadlock.
- `ls | grep txt &`.

Memory and robustness

- `valgrind --leak-check=full --show-leak-kinds=all ./shell` on Linux. On macOS valgrind is unavailable — build with `-fsanitize=address -g` instead, or use `leaks -atExit - - ./shell`.
- Note: memory "still reachable" in exec'd children is normal and not a leak worth chasing; focus on definitely-lost blocks in the shell process.
- Empty input (bare Enter), whitespace-only input, a very long line, `|` alone, `ls |`, `> file` with no command — none of these may crash.

Segfaults are explicitly penalised.

Submission structure

- Directory named exactly `<Roll-No>_OSMiniProject`.
- Contains `README.md`, `makefile`, and all `.c/.h` files **at the top level, no subdirectories**.
- No `.o` files, no executable, no `.git`, no editor backup files, no history file.
- README documents: how to build, what's implemented, any assumptions (e.g. how you treat `&` mid-line, whether `history` records itself), and any known limitations. Stating a limitation honestly reads far better than a grader discovering it.
- `tar -czf <Roll-No>_OSMiniProject.tar.gz <Roll-No>_OSMiniProject/`
- Extract the tarball into a fresh empty directory, run `make`, run the shell. This catches the file you forgot to add and the absolute path you hardcoded.
- If you develop on macOS, test on Linux before submitting — grading is almost certainly on Ubuntu, and a few things differ (`HOST_NAME_MAX`, some `<sys/*.h>` locations, and macOS's stricter `sigaction` prototypes).

One last framing: the marks here concentrate in §9 and §10 not because they're long but because they're where the OS actually shows through. If you can explain out loud why an unclosed pipe write-end hangs the reader, and why `printf` inside a signal handler is unsafe, you understand this project better than the code alone will show.